Deep Reinforcement Learning Control of UAV-Enabled ISAC Systems for Livestock Monitoring

Oluwaseun Odusole[†], Awanthi Jayasundara[†], Engin Zeydan[*], Madhusanka Liyanage[‡], and Pasika Ranaweera[†]
[*]CTTC/CERCA, Castelldefels, Spain
[†]School of Electrical and Electronic Engineering, University College Dublin, Ireland
[‡]NetsLab, School of Computer Science, University College Dublin, Ireland

***Abstract*—Large-area livestock monitoring requires sensing coverage, reliable data delivery, and energy-aware aerial operation under time-varying conditions. This paper develops a sequential deep reinforcement learning (DRL) controller for a four-UAV integrated sensing and communication (ISAC) system simulated in ns-3 with 5G New Radio connectivity. Unlike configuration-level surrogate search, the proposed controller observes sensing, communication, energy, mobility, herd, and mission-time information every 10 s and changes sensing intervals, target speeds, or target altitudes during the same episode. A synchronous Gymnasium–ns-3 interface enforces one state–action–transition handshake per control window. The policy is learned with Double Deep Q-Networks (Double DQN), experience replay, a separate target network, and epsilon-greedy exploration. The implementation exposes a 50-dimensional state and 25 discrete actions and computes throughput, delay, loss, detection, localisation, and energy over non-overlapping control windows. Unit tests verify the Bellman target and replay path, while an ns-3 integration test confirms that sensing, speed, and altitude actions modify live simulation state. A short three-episode experiment is reported only as an implementation smoke test; statistically powered performance claims are reserved for the predeclared multi-seed evaluation protocol provided in this paper.**

***Index Terms*—deep reinforcement learning, Double DQN, integrated sensing and communication, UAV, livestock monitoring, 5G NR, ns-3, Gymnasium, O-RAN**

I. Introduction

Continuous livestock observation over a large farm is difficult when fixed sensors have incomplete coverage and rural connectivity is limited. Unmanned aerial vehicles (UAVs) can move their sensors towards poorly observed areas and relay measurements through a cellular network. Integrated sensing and communication (ISAC) further allows a common aerial platform to sense targets and deliver sensing reports. These benefits introduce a sequential control problem: a short sensing interval may improve observation frequency but consume additional processing and radio energy; increased speed can improve spatial coverage but increases propulsion energy; and altitude changes affect both geometry and radar range.

The earlier version of this study used a Random Forest (RF) regressor to map a complete, fixed simulator configuration to its final objective value. That is a useful surrogate-assisted random-search benchmark, but it is not reinforcement learning: it contains neither state transitions nor temporally ordered actions, Bellman updates, experience replay, or a learned policy that acts during an episode. The present work corrects that methodological issue by formulating and implementing an online sequential Markov decision process (MDP). Deep Q-Networks combine Q-learning with neural function approximation [1], while Double DQN reduces the maximisation bias that can occur when the same estimator both selects and evaluates the next action [2].

The principal contributions are:

- a reproducible ns-3/Gymnasium environment in which every action changes the running UAV simulation and produces exactly one subsequent KPI window;
- a 50-dimensional state, 25-action control space, energy-aware reward, and separate termination and time-truncation conditions;
- a tested Double-DQN implementation with replay memory, online and target networks, Huber loss, and gradient clipping;
- window-specific communication KPIs derived from FlowMonitor deltas, rather than cumulative statistics incorrectly reused at every decision; and
- an evaluation protocol that separates training, validation, and final test seeds and compares frozen policies on identical test seeds.

II. System Model

A. Scenario and UAV Roles

The simulator contains 80 mobile ground targets in an 800×800 m farm, four UAV user equipments, one 5G NR gNB, and a remote monitoring host. The UAVs have surveillance, patrol, rapid-response, and strategic roles with distinct speed and altitude limits. Cow positions are updated every 1 s. Each UAV periodically performs monostatic radar-like sensing and transmits detections and telemetry through the NR network.

UAV motion follows a correlated Gauss–Markov update,

$$\mathbf{v}_i[n] = \alpha_i \mathbf{v}_i[n-1] + (1-\alpha_i)\bar{\mathbf{v}}_i + \sqrt{1-\alpha_i^2}\,\mathbf{w}_i[n], \quad (1)$$

$$\mathbf{p}_i[n] = \mathbf{p}_i[n-1] + \mathbf{v}_i[n]\Delta t_i, \quad (2)$$

where α_i is the memory coefficient and $\mathbf{w}_i[n]$ is a Gaussian innovation. The controller changes a live target speed or altitude; the mobility model then approaches the target while retaining correlated motion.

B. Sensing and Cooperative Localisation

For UAV i and animal k, the three-dimensional slant range is

$$R_{ik}(t) = \|\mathbf{p}_i(t) - \mathbf{p}_k(t)\|_2. \tag{3}$$

The ground target has $z = 0$, so altitude is included in R_{ik}. This detail is essential because the monostatic received power varies as R^{-4}:

$$P_{r,ik}(t) = \frac{P_t G^2 \lambda^2 \sigma}{(4\pi)^3 R_{ik}^4(t)}. \tag{4}$$

A detection occurs when $10\log_{10}(P_{r,ik}/P_n)$ exceeds the configured threshold, with stochastic misses close to the threshold. SNR-weighted fusion combines simultaneous estimates:

$$\hat{\mathbf{p}}_k = \frac{\sum_{i\in\mathcal{U}_k} w_{ik}\hat{\mathbf{p}}_{ik}}{\sum_{i\in\mathcal{U}_k} w_{ik}}, \qquad w_{ik} = 10^{\mathrm{SNR}_{ik}/10}. \tag{5}$$

The sensing KPIs are detection probability P_{det} and localisation root mean-square error R_{mse}.

C. Window Communication KPIs

Let $\Delta B_{i,t}^{\mathrm{rx}}$, $\Delta N_{i,t}^{\mathrm{rx}}$, and $\Delta N_{i,t}^{\mathrm{tx}}$ denote FlowMonitor counter differences for ISAC flow i during decision window t of duration Δ_c. The window metrics are

$$T_t = \frac{1}{|\mathcal{F}|}\sum_{i\in\mathcal{F}} \frac{8\Delta B_{i,t}^{\mathrm{rx}}}{10^6\Delta_c}, \tag{6}$$

$$D_t = \frac{1}{|\mathcal{F}|}\sum_{i\in\mathcal{F}} \frac{\Delta d_{i,t}}{\max(1, \Delta N_{i,t}^{\mathrm{rx}})}, \tag{7}$$

$$L_t = \frac{100}{|\mathcal{F}|}\sum_{i\in\mathcal{F}} \frac{\Delta N_{i,t}^{\mathrm{lost}}}{\max(1, \Delta N_{i,t}^{\mathrm{tx}})}. \tag{8}$$

Only ISAC-report flows are included. Counter snapshots are stored after each decision, preventing cumulative history from being counted as the current transition.

D. Energy Model

The cumulative energy of UAV i is

$$E_i = E_i^{\mathrm{prop}} + E_i^{\mathrm{RF}} + E_i^{\mathrm{proc}}. \tag{9}$$

Propulsion power contains hover and velocity-dependent terms,

$$P_i^{\mathrm{prop}} = P_i^{\mathrm{hover}} + c_v\|\mathbf{v}_i\|^2, \tag{10}$$

RF energy integrates transmit power, and per-sensing-step processing energy is

$$e_i^{\mathrm{proc}} = \kappa(c_0 + c_1 N_i^{\mathrm{det}}) f_i^2. \tag{11}$$

The controller receives the remaining battery fraction for every UAV and an episode terminates if any cumulative energy reaches its budget.

TABLE I
DRL STATE VECTOR ($|\mathcal{S}| = 50$)

State group	Features	Count
Window KPIs	P_{det}, RMSE, throughput, delay, loss	5
Battery	Remaining fraction for four UAVs	4
UAV kinematics	(x, y, z, v_x, v_y, v_z) per UAV	24
Herd summary	Centroid (x, y) and spread (σ_x, σ_y)	4
Live controls	Four sensing intervals	4
	Four target speeds	4
	Four target altitudes	4
Mission phase	Normalised simulation time	1
Total		50

III. SEQUENTIAL DRL FORMULATION

A. Markov Decision Process

At $t = \Delta_c, 2\Delta_c, \ldots$, the environment returns $(\mathbf{s}_t, r_t, \texttt{terminated}, \texttt{truncated})$. Table I defines the state. All values are scaled to bounded numerical ranges; signed horizontal positions and velocities lie in $[-1, 1]$, while the remaining features lie in $[0, 1]$.

The discrete action set contains no-op plus six actions for each UAV role: decrease/increase sensing interval by 0.1 s, decrease/increase target speed by 1 m/s, and decrease/increase target altitude by 5 m. Thus, $|\mathcal{A}| = 1 + 4 \times 6 = 25$. Each value is clipped to its physical role bound.

Using the same normalisation constants as the simulator objective, the window utility is

$$J_t = 3\tilde{P}_{\mathrm{det},t} + 3\tilde{T}_t - 2\widetilde{R}_{\mathrm{mse},t} - \tilde{D}_t - \tilde{L}_t. \tag{12}$$

Let ΔE_t be fleet energy consumed during the window and E_t^{budget} the proportional window budget. The reward is

$$r_t = J_t - \lambda_E \frac{\Delta E_t}{\max(E_t^{\mathrm{budget}}, \epsilon)}. \tag{13}$$

Battery exhaustion sets `terminated`; reaching mission time sets `truncated`. The mission phase in the state makes the finite horizon observable.

B. Synchronous ns-3–Gymnasium Handshake

The C++ simulator opens a local TCP server. After a control window it sends one newline-delimited JSON transition containing episode ID, step ID, state, reward, flags, and raw KPIs. Python validates the IDs and 50-value observation, sends one integer action with matching IDs, and blocks until the next window. This lock-step design prevents stale actions, missing decisions, and ambiguous credit assignment. Episode directories store the full command, KPI log, summary, and simulator output and are immutable to prevent accidental appending or overwriting. The Gymnasium API follows the standard `reset`/`step` interface [4].

C. Double Deep Q-Network

The online network $Q_\theta(\mathbf{s}, a)$ is a multilayer perceptron with two 256-unit ReLU hidden layers and 25 outputs. A replay memory stores $(\mathbf{s}_t, a_t, r_t, \mathbf{s}_{t+1}, d_t)$. For a sampled transition,

TABLE II
IMPLEMENTED DEFAULT PARAMETERS

Parameter	Value
UAVs; cows; field	4; 80; 800×800 m
Mission; control interval	300 s; 10 s
Carrier; NR bandwidth	3.5 GHz; 20 MHz
UE/gNB transmit power	23/38 dBm
Sensing transmit power/gain	30 dBm/20 dB
SNR threshold; position noise	30 dB; 5 m
Initial UAV altitudes	60, 80, 100, 120 m
State/action dimensions	50/25
Hidden layers	256, 256 ReLU
γ; learning rate	0.99; 3×10^{-4}
Replay/batch size	100000/128
Learning starts; target update	2000/1000 steps
Exploration	1.0 to 0.05 over 30000 steps

Double DQN chooses the next action with the online network and evaluates it with the target network $Q_{\bar{\theta}}$:

$$a^* = \arg\max_a Q_\theta(\mathbf{s}_{t+1}, a), \tag{14}$$

$$y_t = r_t + \gamma(1 - d_t) Q_{\bar{\theta}}(\mathbf{s}_{t+1}, a^*). \tag{15}$$

The Huber loss between $Q_\theta(\mathbf{s}_t, a_t)$ and y_t is minimised using AdamW. Gradients are clipped to norm 10. The target weights are copied from the online network every 1000 environment steps. Exploration decreases linearly from $\epsilon = 1.0$ to 0.05 over 30000 steps.

1: Initialise online network Q_θ, target network $Q_{\bar{\theta}} \leftarrow Q_\theta$, and replay memory $\mathcal{D}$
2: **for** each training episode **do**
3: Reset ns-3 with a predeclared training seed and receive $\mathbf{s}_0$
4: **while** episode is active **do**
5: Select random action with probability ϵ; otherwise $a_t = \arg\max_a Q_\theta(\mathbf{s}_t, a)$
6: Apply a_t in live ns-3 and receive $(\mathbf{s}_{t+1}, r_t, d_t)$
7: Store transition in $\mathcal{D}$
8: Sample a mini-batch and update θ using (15)
9: Periodically copy $\bar{\theta} \leftarrow \theta$
10: **end while**
11: **end for**

IV. EXPERIMENTAL METHODOLOGY

A. Simulation and Learning Parameters

Table II reports code defaults, correcting inconsistencies in the earlier manuscript, which listed a 600 s duration and 15 dB threshold while the available implementation used 300 s and 30 dB. Any non-default final setting must be recorded by the generated per-episode command manifest.

B. Baselines and Fair Comparison

The comparison contains: (i) static/no-op control; (ii) uniformly random actions with a recorded RNG seed; (iii) a transparent threshold rule; (iv) the released 28-feature RF surrogate configuration, held static within an episode; and (v) the frozen Double-DQN policy. The RF model is a benchmark and is not described as RL. Its released feature set excludes UE and gNB transmit power, although the earlier thesis counted them when referring to 30 variables.

Training, validation, and test seeds are disjoint. Hyperparameters and checkpoint selection use validation seeds only. After selection, the network is frozen and every method is evaluated on identical unseen test seeds. The primary analysis reports paired per-seed differences, mean, standard deviation, and 95% confidence intervals. Multiple independent training seeds should additionally quantify optimisation variability.

TABLE III
COMPLETED VERIFICATION TESTS

Test	Verified property
Analytic Double-DQN target	Online argmax and target-network value; terminal masking
Replay-memory test	State/action/reward batch shapes and sampling path
Mock-environment learning	Greedy policy learns known preferred action 7
Protocol socket-pair tests	Episode/step IDs, action transfer, 50-state validation
ns-3 integration test	Live sensing, target speed, target altitude, and actual altitude change
Build and syntax checks	ns-3 executable, Python compilation, whitespace checks

TABLE IV
POST-FIX PIPELINE SMOKE TEST ON ONE UNSEEN SEED

Policy	Return	P_{det}	RMSE (m)	Energy (J)[a]
Static	2.6033	0.5609	9.4829	11239.26
Random	2.7903	0.5808	8.9565	11239.32
Rule	2.7215	0.5665	9.1766	11239.30
Double DQN	2.5076	0.5655	9.4867	11239.20

[a] Energy in the final 10 s window. One seed; no CI can be estimated.

V. VERIFICATION AND PRELIMINARY RESULTS

A. Software Verification

The ns-3 integration test completed in 66.9 s. It decreased the surveillance sensing interval, increased its speed target, and increased its altitude target across consecutive decision windows, then verified the corresponding state changes. An inherited range defect was also corrected: the earlier code used the UAV altitude at both range endpoints and therefore removed vertical distance from radar sensing. The corrected calculation measures UAV-to-ground slant range.

B. End-to-End Smoke Test

The post-fix smoke campaign used three 30 s training episodes, producing only six transitions. Candidate checkpoints were evaluated on validation seed 7002; all tied, and the first was frozen. Table IV reports one subsequent matched test seed (9002). These data verify execution and file generation; they are far too small for algorithm ranking.

The smoke DQN does not outperform the baselines, nor should it be expected to: its epsilon remained approximately 1.0 and it received six transitions, below the default 2000-step learning-start threshold used for a real campaign. The smoke run deliberately used a reduced threshold only to exercise

gradient updates. Consequently, Table IV must not appear as evidence that DRL improves the system.

C. Required Final Results

Before submission, replace this subsection with automatically generated tables from the full campaign. At minimum report:

- learning curves across independent agent seeds with validation return;
- final static, RF, random, rule, and frozen-DQN results on the same unseen seeds;
- paired differences and 95% confidence intervals for return, P_{det}, RMSE, throughput, delay, loss, and energy;
- action frequencies and bound-hit rates to show what the policy learned;
- ablations removing energy, communication KPIs, and mobility actions; and
- wall-clock cost and sensitivity to control interval.

[FINAL CAMPAIGN REQUIRED: Insert the multi-seed final table and learning curves generated by the documented full-campaign commands. Do not reuse the legacy 40/14-run RF numbers as DRL results.]

VI. Reproducibility and Limitations

The audited public repository contains one commit, no tags, and no original CSV, model, or command manifest. The supplied thesis PDF contains no embedded data. Therefore, the exact 300-plus-run thesis dataset cannot be reconstructed from the available artifacts. New RF datasets are generated with a fixed sampling seed and a saved configuration and command for each run. Every DRL episode also records its command, seed, KPI windows, simulator log, and final summary. Existing episode directories are never silently overwritten.

The simulator is a controlled abstraction rather than a field validation. Radar returns, animal motion, energy, channel effects, and processing load are modelled; hardware faults, weather, regulatory restrictions, and real animal behaviour are not. The 50-feature state uses herd centroid and spread rather than all animal positions, so the process may be partially observed. Future work should compare recurrent agents, validate models against field data, and deploy only a tested frozen policy through an O-RAN xApp with safety bounds and a fallback controller.

VII. Conclusion

This work replaces a mislabeled RF surrogate search with a genuine sequential DRL controller for UAV-enabled ISAC livestock monitoring. The implementation connects ns-3 and Gymnasium through a strict state–action handshake, applies sensing and mobility actions to live objects, computes communication KPIs per decision window, and trains a tested Double-DQN agent. Unit and integration tests establish software correctness, and a small smoke campaign establishes end-to-end operability. It does not establish a performance advantage. The provided seed-separated, frozen-policy protocol defines the remaining experiment required for a defensible performance claim.

References

[1] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[2] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with Double Q-learning," in *Proc. AAAI*, vol. 30, no. 1, 2016.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[4] M. Towers et al., "Gymnasium: A standard interface for reinforcement learning environments," arXiv:2407.17032, 2024.

[5] F. Liu et al., "Integrated sensing and communications: Toward dual-functional wireless networks for 6G and beyond," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 6, pp. 1728–1767, 2022.

[6] G. F. Riley and T. R. Henderson, "The ns-3 network simulator," in *Modeling and Tools for Network Simulation*. Berlin, Germany: Springer, 2010, pp. 15–34.